Time Complexity 02

Thumrongsak Kosiyatrakul
tkosiyat@cs.pitt.edu

Polynomial Time vs Exponential Time

- Polynomial time is considered small but exponential time is considered large
- Consider a problem size $n = 1000$
  - Assume that algorithms A and B requires to execute n^2 and 2^n instructions respectively
  - Assume that each instruction takes 1 nanosecond
- Algorithm A takes

$$1000^2 \times 10^{-9} = 10^6 \times 10^{-9} = 10^{-3} = 0.001 \text{second}$$

- Algorithm B takes

$$2^{1000} \times 10^{-9} \approx 10^{301} \times 10^{-9} = 10^{292} \text{ seconds } \approx 10^{284} \text{ years}$$

Class P

Definition 7.12

P is the class of languages that are decidable in polynomial time on a deterministic single-tape Turing machine. In other words,

$$P = \bigcup_k \mathrm{TIME}(n^k)$$

- Recall that $\mathrm{TIME}(n^k)$ is a set of languages that can be decided in $\mathcal{O}(n^k)$ time Turing machine
- P is a class of problems that are realistically solvable on a computer

Class P

- Consider the following Turing machine:

  $M =$ "On input x:

  1. Do this
  2. While this is true
  3. Do that

- There are two meanings of the word "step" of a TM M:
  - The above TM has three steps, step 1, step 2, and step 3
  - Each step takes a number of steps to execute
    - One step results in a configuration
- A TM runs in **polynomial time** if
  1. The number of times each step is executed (including repeated) is a polynomial (n^k for some k)
  2. Each step takes a polynomial time ($\mathcal{O}(n^k)$ for some k)

Analyzing Algorithms

- Recall our Turing machine M_1 that decides

$$A = \{0^k 1^k \mid k \geq 0\}$$

$M_1 =$ "On input string w:

1. Scan across the tape and *reject* if a 0 is found to the right of a 1.
2. Repeat if both 0s and 1s remain on the tape:
3. Scan across the tape, crossing off a single 0 and a single 1.
4. If 0s still remain after all the 1s have been crossed off, or if 1s still remain after all the 0s have been crossed off, *reject*. Otherwise, if neither 0s nor 1s remain on the tape, *accept*."

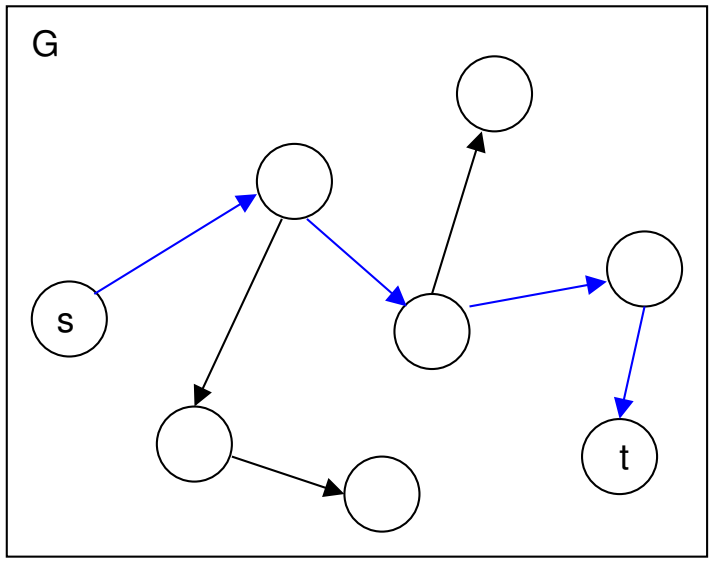
Analyzing Algorithm

- Input 0^k1^k where $k = n/2$
  - Number of times each step is executed:
    - Step 1 will be executed 1 time
    - Step 2 and 3 will be executed $n/2$ times
    - Step 4 will be executed 1 time
  - Number of steps of each step
    - Step 1 takes $\mathcal{O}(n)$ steps
    - Step 2 and 3 take $\mathcal{O}(n)$ steps
    - Step 4 takes $\mathcal{O}(n)$ steps
- From the analysis:
  - The number of times each step is executed (including repeat) is polynomial
  - Each step takes polynomial time
  - Therefore, M_1 runs in polynomial time

Examples of Problems in P

- Let

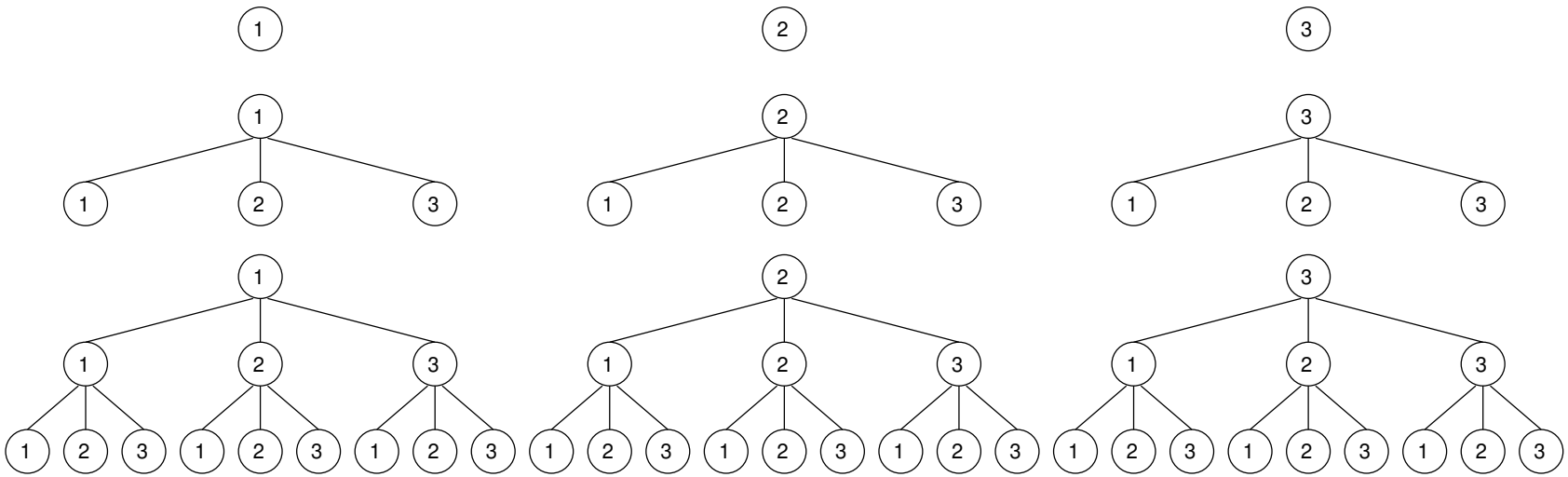
$$PATH = \{\langle G, s, t\rangle \mid G \text{ is a directed graph that has a directed path from } s \text{ to } t\}$$



- Is $PATH \in P$?
  - If $PATH$ can be decided by a TM in polynomial time, $PATH \in P$

Brute-Force Algorithm for *PATH*

- Suppose a directed graph G consists of m nodes
  - A brute-force algorithm:
    - Creating all possible paths of length at most m
    - Search for a direct path from s to t
- Example: All possible paths of length at most 3:



Maximum Paths with 3 nodes

Brute-Force Algorithm for *PATH*

- The number of possible paths is roughly m^m
- For each path, you need to check whether it is a path from s to t
- There are roughly m^m paths
- To search all possible paths is $\mathcal{O}(m^m)$

$$\mathcal{O}(m^m) = \mathcal{O}((2^{\log_2 m})^m) = \mathcal{O}(2^{m \log_2 m})$$

- This will take an exponential time

Breadth-First Search for *PATH*

- Use breadth-first search to achieve polynomial time for *PATH*:

  $M =$ "On input $\langle G, s, t \rangle$, where G is a directed graph with nodes s and t:

  1. Place a mark on node s.
  2. Repeat the following until no additional nodes are marked:
  3. Scan all the edges of G. If an edge (a, b) is found going from a marked node a to an unmarked node b, mark node b.
  4. If t is marked, $accept$. Otherwise, $reject$."

- For simplicity of analysis, we are going to assume the following:
  - The number of nodes is n
  - The number of edges is proportional to the number of nodes kn edges for some k
  - The string representation of a graph is proportional to the number of nodes and the number of edges
- In doing so, we can simply use n (number of nodes) as the problem size instead of the length of the input string

Breadth-First Search for *PATH*

- Analysis where input string is $\langle G, s, t \rangle$ where G is a directed graph and s and t are nodes
  - Step 1: Place a mark on node s.
    - This step will be executed one time and the number of steps is $\mathcal{O}(n)$
  - Step 4: If t is marked, $accept$. Otherwise, $reject$.
    - This step will be executed one time and the number of steps is $\mathcal{O}(n)$
  - Steps 2 and 3:
    - Runs at most n times (mark one additional node every repetition)
    - Recall that there are kn edges
    - For each execution of step 3, it needs $\mathcal{O}(kn) = \mathcal{O}(n)$ steps
- Conclusion:
  - Number of times each step will be executed is polynomial
  - Each step is polynomial
  - Therefore, this algorithm M is polynomial

Relatively Prime

- Two numbers are **relatively prime** if 1 is the largest integer that evenly divides them both
- Formally,

$$RELPRIME = \{\langle x, y \rangle \mid x \text{ and } y \text{ are relatively prime}\}$$

- Is $RELPRIME \in P$?
- Brute-force algorithm:
  - Find all possible divisors of both numbers and accept if none are greater than 1
  - If a number is represented by n-bit binary, the number of possible value is 2^n
  - Thus, to find all possible divisor of x and y is $\mathcal{O}(2^n)$ (exponential)

Euclidean Algorithm for Greatest Common Divisor

- We can check whether x and y are relatively prime by computing their Greatest Common Divisor (GCD)
  - If $gcd(x, y) = 1$, x and y are relatively prime
- Computing a gcd can be done using Euclidean Algorithm in a form of a TM as follows:

  $E =$ "On input $\langle x, y \rangle$, where x and y are natural numbers:

  1. Repeat until $y = 0$:
  2. Assign $x \leftarrow x \mod y$.
  3. Exchange x and y.
  4. Output x."
- When we run TM E on input $\langle x, y \rangle$ where x and y are natural numbers, it simply output their GCD

Euclidean Algorithm for Greatest Common Divisor

- This algorithm R solves $RELPRIME$

  $R =$ "On input $\langle x, y \rangle$, where x and y are natural numbers in binary:

  1. Run E on $\langle x, y \rangle$.
  2. If the result is 1, $accept$. Otherwise, $reject$.

- Analysis:
  - There is no loop in TM R
    - Step 1 will be executed one time
    - Step 2 will be executed one time
  - For the number of steps in each step:
    - For step 1, it depends on the number of steps of TM E
    - For step 2, at most $\mathcal{O}(n)$

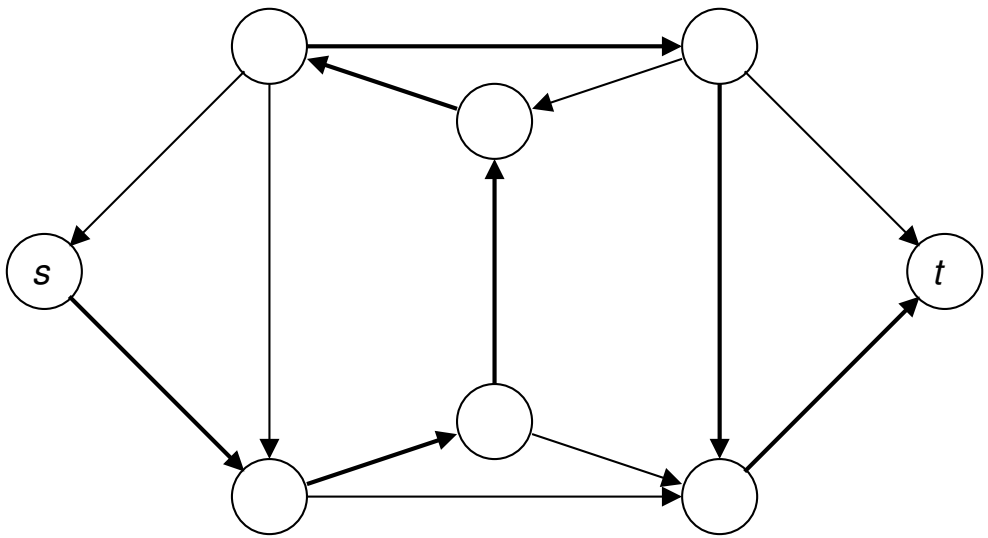
$RELPRIME \in P$

- Analysis of Algorithm E:
  - Given two numbers x and y, there are two possibilities:
    1. $x/2 \geq y$ or
    2. $x/2 < y$
  - If $x/2 \geq y$, $x \bmod y < y < x/2$
  - If $x/2 < y$, $x \bmod y = x - y < x/2$
  - In other words, performing $x = x \bmod y$ reduces the value of x roughly in half
  - Thus, steps 2 and 3 will be repeated roughly $2 \log_2 x$ or $2 \log_2 y$ (proportional to the length of either x or y)
    - Multiplied by 2 because of the swap in step 3
  - Algorithm E runs in polynomial time
- $RELPRIME \in P$

Hamiltonian Path

- A **Hamiltonian path** in a directed graph G is a directed path that goes through each node exactly once
- Formally,

$$HAMPATH = \{\langle G, s, t\rangle \mid G \text{ is a directed graph with a Hamiltonian path from } s \text{ to } t\}$$



- Is $HAMPATH \in P$?

Polynomial Verifiability

- Unfortunately, we need the brute-force algorithm:
  - First, we generate all possible paths
  - Then check each path whether it is a Hamiltonian path from s to t
- This requires exponential time since there are roughly m^m possible paths
- No one know an algorithm that can solve Hamiltonian path in polynomial time yet
- No one can **prove** that the Hamiltonian path problem **cannot be solved in polynomial time** yet

$PATH$ vs $HAMPATH$

- Recall languages $PATH$ and $HAMPATH$

$$PATH = \{\langle G, s, t\rangle \mid G \text{ is a directed graph that has a directed path from } s \text{ to } t\}$$

$$HAMPATH = \{\langle G, s, t\rangle \mid G \text{ is a directed graph with a Hamiltonian path from } s \text{ to } t\}$$

- Given $\langle G, s, t\rangle$:
  - to decide whether it is in $PATH$ can be done in polynomial time
  - to decide whether it is in $HAMPATH$ (as of now) cannot be done in polynomial time

COMPOSITES

- Another problem is called $COMPOSITES$
- Formally,

$$COMPOSITES = \{x \mid x = pq, \text{ for integers } p, q > 1\}$$

- Given a composite number $x = pq$ where p and q are large prime numbers, it takes a very long time to find p and q
- The RSA algorithm is based on the fact that $COMPOSITES$ requires an exponential time